Spatial Data Science (SDS)

Spatial Data Science (SDS):
- descriptive - where things happen, spatio-temporal relations,
- perspective - why things happen there, make predictions

Spatial Data (SD) , a.k.a. Geospatial data, GIS data, or geodata
SD consists of two components:
- what (attribute data - nominal, ordinal, interval, ratio)
- where (spatial data) – **coordinates, topology, locations, shapes**

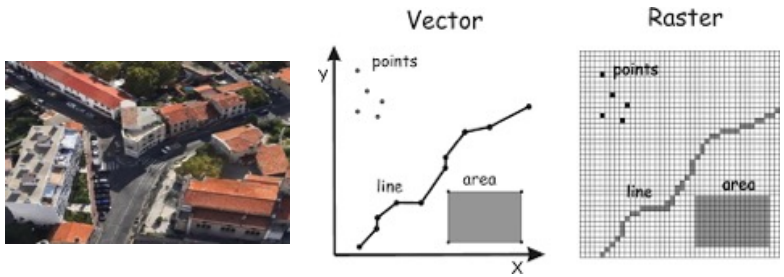
Example of SD - a road map: a 2-dim object that contains **points**, **lines**, and **polygons** that can represent cities, roads, boundaries (of states, provinces, ...).

GIS (Geographic Information System) - used to store, retrieve, and render the Earth-relative SD. Examples - Canadian GIS, U.S. Census DIME files, POLYVRT system - use Polygonal representations. Surface data has been represented by the triangle structures.

SD can be of 2 types:
- **raster data** (grid of rows/columns)
- **vector data** (points, polylines, and polygons).

SD – data collection:
- Satellites (spatial and spectral resolution, time resolution), geostationary orbit satellites
- Aerial photography (low-altitude) using unmanned aerial vehicles and delta-winged drones.
- Ground photography, streetscape scanning
- Ground boring data - resources (water, oil, etc.)
- Political and ownership boundaries, geo-political, languages, demographics, etc.
- Nature – plants, animals, environmental conditions, land classification for aggriculture, ...
- roads, rail-roads, airplane routes
- pipes, internet and cables, cellular data, tracking people movements and travel
- GPS, Google map, drive routing
- Geo-marketing, geo-sales predictions, spatial propensity for products/services, voting, criminal behavior, religion, ...
- Follow the money, real estate planning,

Good review:
https://www.sciencedirect.com/topics/engineering/spatial-data



Geo-statistics - analysis of spatial continuity and weak stationarity.
Point process - model for a spatial distribution of points Example - positions of trees in a forest (comparing with a random pattern).
Lattice - model a set of (ir)regular shapes with neighborhood relation.
SAR = Spatial Auto-Regression analysis
MRF = Markov Random Field analysis
DEM = Digital Elevation Model
SAC = Spatial Analysis Center

SDSC – SDS Conference - https://spatial-data-science-conference.com/

ESRI ArcGIS

ESRI = Environmental Systems Research Institute. Since 1969. 1+ Billion revenue, ~4K employees in 73 countries, 49 offices worldwide. GIS Mapping, Location Intelligence, Spatial Analytics.
- https://en.wikipedia.org/wiki/Esri
- https://www.esri.com/en-us/about/about-esri/who-we-are
- https://www.esri.com/

ArcGIS - GIS software (online and desktop) provided by ESRI (free, student, commercial options).
Arc = "arc" in geometry, GIS = Geographical Information System
- https://en.wikipedia.org/wiki/ArcGIS
- https://www.esri.com/en-us/arcgis/about-arcgis/overview
- https://www.esri.com/en-us/arcgis/products/index
- https://www.esri.com/en-us/arcgis/products/arcgis-notebooks/overview - ArcGIS Notebooks
- https://github.com/Esri - about 600 repositories
- https://www.youtube.com/watch?v=UL6atkzPvsw - Spatial Data Science - A Tour
- https://desktop.arcgis.com/en/documentation/ - documentation
- https://www.youtube.com/watch?v=BbUctneHfKc – ArcGIS tutorial (part 1 of series)

Spatial Data Science (SDS) Course Syllabus

Section 1
Introduction to Spatial Data Science (SDS)
- spatial data, tools, and analysis
- place-based context can reveal hidden patterns in data
- use ArcGIS Pro and ArcGIS Notebooks.

Section 2
The Spatial Approach to Predictive Analysis
- add spatial properties into modeling workflows to add predictive power.
- Use Random Forest to train the model and make predictions.

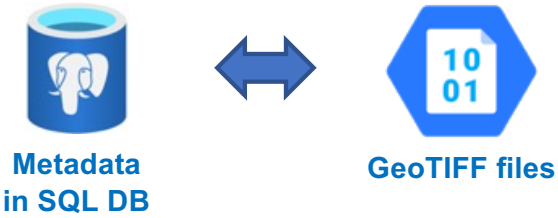
Section 3
Finding Optimal Locations Using Suitability Models
- Where is the best location for <fill in the blank>?
- Perform a weighted overlay analysis, rank multiple suitability criteria.
- Learn to transform data using functions to better represent suitability impact.

Section 4
Pattern Detection and Clustering
- ArcGIS tools to identify patterns and clusters
- statistical clustering - analyze patterns in space and time.
- Create a spacetime cube, explore spatio-temporal trends, find where and when high/low clusters occur.

Section 5
Object Detection with Deep Learning
- Prepare training sample data,
- Use a neural network to train an object detection model.

Section 6
Communicating Results with Impact
- Be effective and visual storyteller.
- Present facts using information products that resonate with your audience, make decision-makers to understand and act on it.

PostGIS
Spatial Database



PostGIS - spatial database extender for PostgreSQL
(since 2001)
- https://postgis.net/
- https://postgis.net/stuff/postgis-3.1.pdf (874p. manual)
- https://en.wikipedia.org/wiki/PostGIS

Great lecture by Paul Ramsey:
- https://www.youtube.com/watch?v=g4DgAVCmiDE

Azure **PostgreSQL** flexible server
- https://docs.microsoft.com/en-us/azure/postgresql/flexible-server/

Good advisor - Dan Morris, MicrosoftAI for Earth Program Director
(also Rob on his team)
- https://github.com/microsoft/AIforEarthDatasets

GIS without GIS

The essence of ESRI Image Server:
store the data about the geometry of the images
in a big metadata store (SQL database)
(with pointers to blob storage).

You can achieve similar functionality
by using a managed database with geospatial extensions
that will let you run spatial/temporal queries over geospatial metadata:

PostgreSQL + PostGIS:
- https://azure.microsoft.com/en-us/services/postgresql/

MS SQL Server:
- https://docs.microsoft.com/en-us/sql/relational-databases/spatial/spatial-data-sql-server

Oracle Spatial:
- https://www.oracle.com/database/spatial/

GeoTIFF

You typically you don't need to read the entire **TIFF** file when doing analysis.
Better approach is to use cloud-optimized **GeoTIFF** format:
- http://cogeo.org .

This is still a proper **TIFF** file, but its header contains
"table of contents" for the file (which areas of the image are in what parts of the file).

You can typically convert **GeoTIFF** to **COG** with **GDAL**:
- https://gdal.org/drivers/raster/cog.html

CARTO, QGIS, Google Earth

CARTO

- https://carto.com/ - Location Intelligence platform allows organizations to store, enrich, analyze & visualize their data to make spatially-aware decisions.
- CARTO uses Google BigQuery
- **CARTOframes** - **Python** package for integrating CARTO maps, analysis, and data services into data science workflows.
  - https://github.com/CartoDB/cartoframes
- Pricing – limited 12 month free plan, normally $200/month (or more)

QGIS

A Free and Open Source GIS (Geographic Information System)
- https://www.qgis.org/en/site/ -

Google Earth Engine

multi-petabyte catalog of satellite imagery and geospatial datasets
with planetary-scale analysis capabilities
allows to detect changes, map trends, and quantify differences on the Earth's surface
- https://earthengine.google.com/

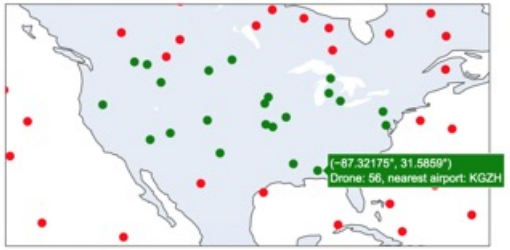
Hadoop >> Spark >> PySpark >> GeoSpark

Spark was initially started by Matei Zaharia at UC Berkeley's AMPLab in 2009,
and open sourced in 2010 under a BSD license.
In 2013 the project was donated to the Apache Software Foundation, Databricks was formed

- https://logz.io/blog/hadoop-vs-spark/
- https://en.wikipedia.org/wiki/Apache_Spark
- https://en.wikipedia.org/wiki/Databricks
- http://spark.apache.org/docs/latest/ - documentation and tutorials

Matei Zaharia
Spark
Databricks

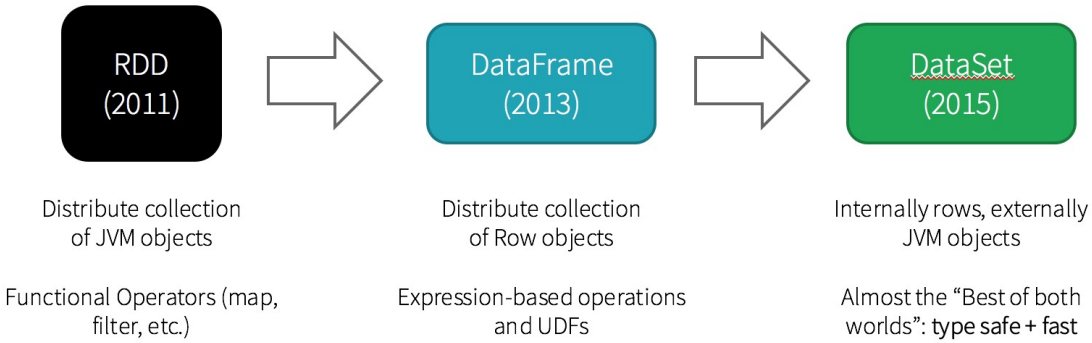
Distribute and process large spatial data across multiple computers



GeoSpark (Spatial RDDs and Spatial SQL)
- https://github.com/apache/incubator-sedona
- https://pypi.org/project/geospark/ - Python wrapper on GeoSpark library
- https://cran.r-project.org/web/packages/geospark/index.html - R-library
- https://cloudarchitected.com/2019/07/geospatial-analytics-in-databricks-with-python-and-geomesa/

- https://www.linkedin.com/pulse/automating-webgis-azure-databricks-andy-tangeman-gisp/ - Azure Databricks for Web GIS
- https://rasterframes.io/ - DataFrame-centric view over geospatial raster data, spatiotemporal queries, map algebra raster operations, and interoperability with Spark ML
- https://www.geomesa.org/ - Store, index, query, and transform spatio-temporal data at scale in HBase, Accumulo, Cassandra, Redis, Kafka and Spark
- https://www.opengeohub.org/machine-learning-spatial-data - a course

PySpark – Working with Data



Central idea in Spark is parallel processing of data by multiple compute nodes of the cluster.

There are three main distributed data structures in Spark:

- **Spark RDD** (Resilient Distributed Dataset) - immutable (Read-only) collections of objects of varying types, which computes on the different nodes of a given cluster
- **Spark DataFrame** (distributed, not a pandas one) - distributed collection of data organized into named columns. Like a table in a relational database
- **Spark Dataset** - strongly typed, represents queries with encoders (Java, Scala)

Parquet

APACHE ARROW

Apache Parquet vs Apache Arrow

Apache Parquet - free open-source column-oriented data storage format of the **Apache Hadoop** ecosystem. It uses compression. Designed for disk storage.

Apache Arrow – protocol to store DataFrames in memory and on disk. Designed for "in-memory".

Parquet files are often much smaller than **Arrow**-files.

- https://parquet.apache.org/
- https://arrow.apache.org/

Wes McKinney

Feather format
- https://arrow.apache.org/docs/python/feather.html

Azure and Geo Spatial

Azure Maps
Azure Maps is the natural product for showing spatial information on a map in an Azure-based architecture (other than Esri tools, which can also be a great solution).
- https://azure.microsoft.com/en-us/services/azure-maps/

Azure Cosmos DB
CosmosDB also has a set of spatial features that would get the same thing done (as MS SQL Server or PostgreSQL):
- https://docs.microsoft.com/en-us/azure/cosmos-db/sql-query-geospatial-intro
- https://towardsdatascience.com/geospatial-big-data-performance-tests-with-cosmos-db-and-data-enrichment-with-azure-synapse-257db7d29e41

Azure Stream Analytics – GeoSpatial Functions
- https://azure.microsoft.com/fr-fr/blog/new-in-azure-stream-analytics-geospatial-functions-custom-code-and-lots-more/

Azure GeoAI DSVM
Data Science Virtual Machine was introduced in 2018
- https://azure.microsoft.com/en-us/blog/microsoft-and-esri-launch-geospatial-ai-on-azure/

Azure ArcGIS Enterprise Server (ESRI)
- https://azure.microsoft.com/en-us/blog/microsoft-and-esri-launch-geospatial-ai-on-azure/

Azure Distributed Solutions
using Deep-Learning and GPU-enabled servers (2019)
- https://medium.com/geoai/integrating-deep-learning-with-gis-70e7c5aa9dfe

Tools

- python, numpy, pandas
- **GeoPandas** - https://geopandas.org/ - open source, adds datatypes to allow spatial operations on geometric types (using shapely).
- Shapely - https://shapely.readthedocs.io/en/stable/ - manipulation and analysis of planar geometric objects
- Fiona – https://fiona.readthedocs.io/en/latest/ - read/write geo-data, modules to use **GDAL** (Geospatial Data Abstraction Library).
- descartes – https://pypi.org/project/descartes/ - Use Shapely or GeoJSON-like geometric objects as matplotlib paths and patches
- matplotlib - https://matplotlib.org/ - plotting
- geoplot - https://residentmario.github.io/geoplot/index.html - high-level Python geospatial plotting library
- scikit-learn - https://scikit-learn.org – very popular open-source Python library for Machine Learning
- SciPy - https://www.scipy.org/ - open-source Python library for math, science, and engineering
- RTree - https://toblerity.org/rtree/ - a ctypes Python wrapper of libspatialindex for advanced spatial indexing features
- GDAL - https://gdal.org/ - translator library for raster and vector geospatial data formats
- PySAL - https://pysal.org/ - Python Spatial Analysis Library
- Missingno - https://github.com/ResidentMario/missingno - missing data visualization module for Python
- GEOJSON - https://geojson.org/ , https://tools.ietf.org/html/rfc7946 - open standard format (JSON) for representing simple geographical features and their non-spatial attributes. The features include points, line strings, polygons, and multi-part collections of these types.
- geojson.io - http://geojson.io/ - a quick, simple tool for creating, viewing, and sharing maps. Supports GeoJSON, also accepts KML, GPX, CSV, GTFS, TopoJSON, and other formats.
- PostGIS - https://postgis.net/ - open source software (extension) which adds support for geographic objects to the PostgreSQL database.
- Sequelize - https://sequelize.org/ - a promise-based Node.js ORM for Postgres, MySQL, MariaDB, SQLite and Microsoft SQL Server.

- deep learning
- spatial clustering, temporal clustering, predictions
- optimization problems (shortest path, etc.)

- Problem - combining different data sources, converting to the same spatial support format.

- **Gogle Earth Engine** - https://earthengine.google.com/ - multi-petabyte catalog of satellite imagery and geospatial datasets with planetary-scale analysis capabilities and makes it available for scientists, researchers, and developers to detect changes, map trends, and quantify differences on the Earth's surface

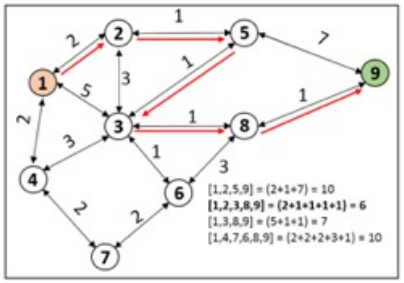
Related Resources

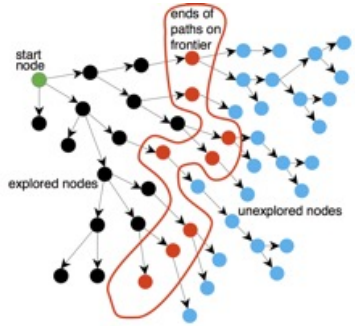
- Tutorial - Introduction to Geospatial Data in Python
  - https://www.datacamp.com/community/tutorials/geospatial-data-python
- Eath Data Analytics Online Certificate
  - https://www.earthdatascience.org/courses/use-data-open-source-python/intro-vector-data-python/spatial-data-vector-shapefiles/
- python-geospatial – a collection of Python packages for geospatial analysis with binder-ready notebook examples
  - https://github.com/giswqs/python-geospatial
- Visualizing geospatial data using python
  - https://towardsdatascience.com/visualizing-geospatial-data-in-python-e070374fe621
- ATLIST – create custom Google maps
  - https://www.atlistmaps.com/
- Course: Python for Spatial Data Analysis with Earth Engine and QGIS
- - https://www.udemy.com/course/python-for-spatial-data-analysis-with-earth-engine-and-qgis/
- ---

How Navigation Apps work ? (Waze, etc.)

It is the **Shortest Path Problem**
- https://en.wikipedia.org/wiki/Shortest_path_problem

See also:
- Dijkstra's algorithm
  - https://en.wikipedia.org/wiki/Dijkstra%27s_algorithm
- A* search algorithm
  - https://en.wikipedia.org/wiki/A*_search_algorithm
- How GPC calculates routes (2011, basically A* algorithm)
  - http://blog.kdgregory.com/2011/12/how-gps-calculates-routes.html





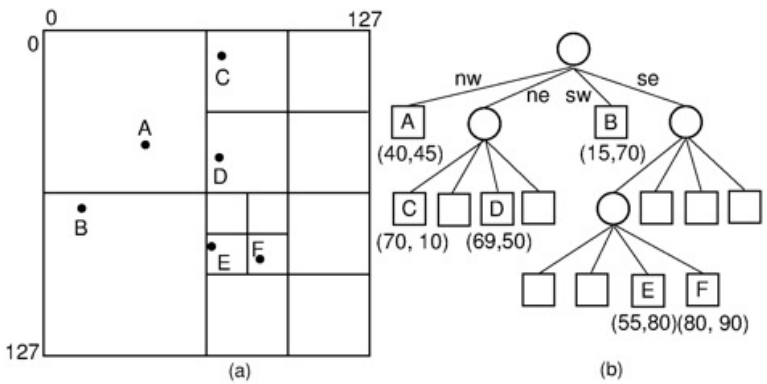
Search - Spatial Data Structures

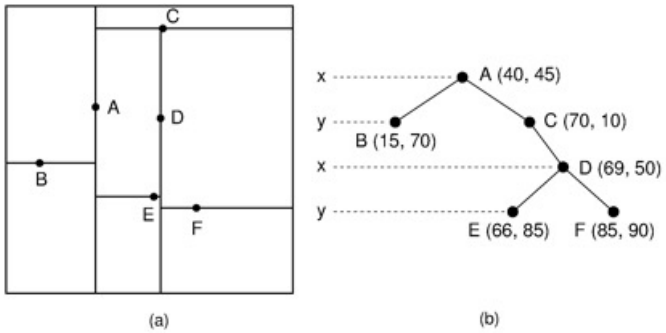
- http://courses.cs.vt.edu/cs3114/Fall08/Quadtree.pdf

Problem – find objects within 10 miles radius.
This is a search with a multidimensional search key (two coordintes),
a.k.a. **Multidimensional Range Query**.
Another popular search - **Nearest Neighbor Search**

To make queries effective we can store spatial data as a tree.
Two common tree structures:

- **k-d tree** - a binary tree whose splitting decisions alternate among the key dimensions.
  - https://en.wikipedia.org/wiki/K-d_tree
- **PR quadtree** (Point-Region Quadtree) - split a square into four equal-sized quadrants at each branch (hence the name "quadtree")
  - https://en.wikipedia.org/wiki/Quadtree





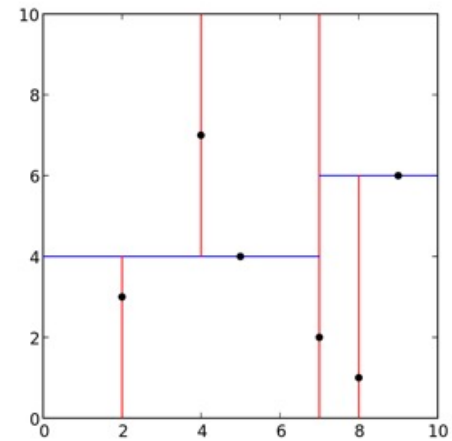
Several variants of **balanced k-d trees**:

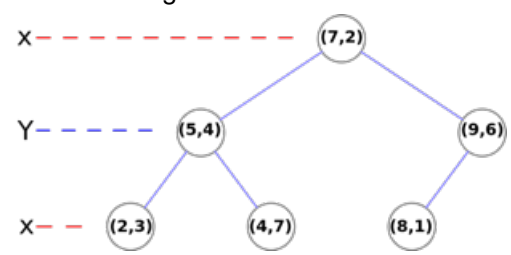
- divided k-d tree
- pseudo k-d tree
- K-D-B-tree
- hB-tree
- Bkd-tree

Many of these are **adaptive k-d trees**.

Search - Building k-d tree - recursively

```python
from collections import namedtuple
from operator import itemgetter
from pprint import pformat

class Node(namedtuple("Node", "location left_child right_child")):
    def __repr__(self):
        return pformat(tuple(self))

def kdtree(point_list, depth: int = 0):
    if not point_list:
        return None

    k = len(point_list[0])  # assumes all points have the same dimension
    # Select axis based on depth so that axis cycles through all valid values
    axis = depth % k

    # Sort point list by axis and choose median as pivot element
    point_list.sort(key=itemgetter(axis))
    median = len(point_list) // 2

    # Create node and construct subtrees
    return Node(
        location=point_list[median],
        left_child=kdtree(point_list[:median], depth + 1),
        right_child=kdtree(point_list[median + 1 :], depth + 1),
    )

def main():
    """Example usage"""
    point_list = [(7, 2), (5, 4), (9, 6), (4, 7), (8, 1), (2, 3)]
    tree = kdtree(point_list)
    print(tree)

if __name__ == "__main__":
    main()

Output:
((7, 2),
 ((5, 4), ((2, 3), None, None), ((4, 7), None, None)),
 ((9, 6), ((8, 1), None, None), None))
```



***k*-d tree** decomposition for the point set (2,3), (5,4), (9,6), (4,7), (8,1), (7,2).

The resulting **k-d tree**